Università di Genova
DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

Metrics analysis to identify anomalous releases in the NPM ecosystem's Software Supply Chain

Michele Frattini

Master's Thesis

Università di Genova, DIBRIS
Via Dodecaneso, 35 16146 Genova, Italy
https://www.dibris.unige.it/

Computer Science MSc
Software Security and Engineering Curriculum

Metrics analysis to identify anomalous releases in the NPM ecosystem's Software Supply Chain

Michele Frattini

Advisor: Giovanni Lagorio

Examiner: Luca Caviglione

January, 2026

Table of Contents

Chapter 1

Introduction

The open-source software **OSS!**s (**OSS!**s) ecosystem represents one of the fundamental pillars of modern software development, with millions of projects relying on publicly shared libraries and packages.

Within this context, the Node Package Manager **NPM!**s (**NPM!**s) ecosystem for JavaScript emerges as one of the largest and most dynamic software repositories, hosting more than two million packages [Zah+22].

This strong interdependence creates complex software supply chains, in which the security and integrity of a project are inherently tied to those of all its direct and transitive dependencies [Guo+24]. As a result, the compromise of a single widely used library can rapidly propagate, causing a cascading effect across many projects.

A significant example of a supply chain attack occurred on September 8, 2025, when an attacker compromised 18 popular **NPM!**s packages, including chalk and color [Hh25]. These packages alone account for over 2.6 billion downloads each week, making this attack one of the most impactful in the recent history of the **NPM!**s ecosystem.

To address these threats, several detection tools have been proposed, mainly based on intrinsic source code analysis, such as DONAPI [Hua+24a] and SpiderScan [Hua+24b].

This thesis proposes an alternative approach to the problem. The goal is to evaluate whether it is possible to identify anomalous updates in an **NPM!**s package using specifically designed metrics, analyzing the evolution of the code over all releases. Unlike approaches that focus on isolated updates, this work considers the entire evolutionary history of a package.

To achieve this, a multi-phase process was followed.

First, two real attacks that occurred within the **NPM!**s ecosystem were selected and used

as case studies for threat modeling. Based on the analysis of these attacks, a set of metrics was defined and classified by type, as they were considered relevant for the identification of these kinds of threats.

Subsequently, a tool was developed to monitor the evolution of these metrics across all versions of an **NPM!**s package.

The tool was initially tested on a small dataset of malicious packages, all related to the threat modeling performed in the previous phase. These packages were identified and downloaded using a custom developed script. This phase made it possible to observe the behavior of the metrics in the presence of malicious code.

A study of the real-world context was also conducted.

In particular, 500 **NPM!**s packages selected from the top 1000 most popular ones were analyzed, in order to understand the behavior of the metrics in real scenarios. This allowed the estimation of the average values assumed by each metric.

By comparing these values with those observed in malicious packages, it was possible to define meaningful threshold values for each metric. Unlike other tools, the thresholds were not defined a priori, as they can be easily bypassed by an attacker, but instead were derived from studying the real-world behavior of the metrics.

Finally, specific behaviors of the metrics to be monitored have been defined, defined as flags, and the corresponding thresholds were associated with each of them.

After validation on the malicious dataset, the complete tool was applied again to the 1000 most downloaded **NPM!**s real-world packages, to evaluate its overall effectiveness.

Chapter 2

Background

Third-party packages play a crucial role in modern software development by reducing implementation effort and providing reusable functionalities that would otherwise be time-consuming to develop from scratch [Sae+25].

To support JavaScript developers in the reuse of third-party code, the **NPM!**s provides hundreds of thousands of free and reusable code software packages. The **NPM!**s platform consists of an online registry for searching packages suitable for given tasks, and a package manager that automatically resolves and installs dependencies, simplify the integration of these packages into projects [Zim+19]. By design, The **NPM!**s ecosystem is open, allowing any user to freely publish.

This openness has been a key factor in the rapid growth and success of **NPM!**s, but it also introduces significant security risks. In particular, the widespread use of third-party dependencies makes **NPM!**s a central component of the JavaScript software supply chain.

The software Supply Chain **SSC!**s (**SSC!**s) refers to the set of processes to select and obtain software components from third parties; it also comprehends the companies involved in these processes [GAH23].

Modern software systems heavily rely on **SSCs!**s (**SSCs!**s), often incorporating a large number of direct and transitive dependencies. Within this context, the **NPM!**s ecosystem exemplifies how supply chain dependencies can amplify the impact of security incidents.

Highly popular packages can directly or indirectly influence many other projects, sometimes exceeding 100,000 dependent packages [Zim+19]. Moreover, installing an average **NPM!**s package implicitly introduces trust on 79 third-party packages and 39 maintainers [Zim+19].

As a consequence, the **NPM!**s ecosystem has become a prominent target for Software

Supply Chain attacks **SCCA!**s (**SCCA!**s), characterized by the injection of malicious code into legitimate packages update, in order to compromise dependent systems further down the chain [Ohm+20].

Chapter 3

Related Work

Several works have done in identification of malicious packages within the **NPM!**s ecosystem, but none of them considered the evolution of the code with respect to all the published releases.

One such tool is DONAPI [Hua+24a], which used a combination of static analysis, based on a predefined database of sensitive **APIs!**s (**APIs!**s); and dynamic analysis, to extract behavioral patterns in order to detect and classify malicious packages.

An alternative tool is SpiderScan [Hua+24b], which is based on graph-based behavior modeling and matching. It uses **LLM!**s (**LLM!**s) for various tasks, including to identification of sensitive **APIs!**s and the analysis of shell commands.

Other tools also use **LLM!**s for **NPM!**s package analysis, such as the tool by Zahan et al. [Zah+25], Wang et al. [Wan+25], and Zhang et al. [Zha+25].

Although these tools achieve good results in identifying malicious packages updates, they are not open source and they also have limitations.

The dynamic analysis in DONAPI, according to Huang et al. [Hua+24b], risks "hindering their practical usefulness in real-world scenarios".

Additionally a common issue, is that most existing tools do not model malicious behavior in a holistic way. In fact, these tools often focus only on predefined set of discrete features, like sensitive **APIs!**s, code structures or semantics (e.g., data flow) [Hua+24b]; but not considering the history of the package code evolution through all its releases. These tools also require a huge amount of manual validation due to high number of false positives.

Furthermore, as discussed by Huang et al. [Hua+24b], "Prompt engineering methods leverage the capability of ChatGPT, and use self-refinement and chain-of-thought techniques to detect maliciousness, but can incur high expense in real-world scenarios", besides the

fact that in general **LLM!**s can be prone to hallucination, as he states Li et al. [Li+24].

Chapter 4

Case Study and Threat Modeling

As a first step, two real-world attacks that occurred within the **NPM!**s Software Supply Chain (**SSC!**s) were selected and analyzed.

The study of these cases made it possible to gain a deeper understanding of the main characteristics of these attacks, providing the basis for the subsequent definition of the metrics necessary to identify these types of threats, in the **??** (**??**).

4.1 Crypto related attack - September 2025

The first attack, which occurred on September 8 2025, compromised 18 highly popular **NPM!**s packages, including `chalk`, `debug`, and `ansi-colors` [Hh25].

The attack was made possible by compromising the account of a maintainer of one of the packages via a phishing email.

After obtaining the account credentials, the attacker published malicious versions of the affected packages, which contained malware designed to steal cryptocurrencies by intercepting and manipulating users' browser transactions.

The malware's operation can be divided into these main phases [Mat25]. First, the malicious code checked for the presence of a Web3 wallet interface in the browser (e.g., `window.ethereum`).

If present, it hooked one of these specific functions (`eth_sendTransaction`, `solana_signTransaction`, `solana_signAndSendTransaction`) with the goal of replacing the transaction's destination address with one controlled by the attacker; this is done by intercepting network calls (using the `fetch` or `XMLHttpRequest` **APIs!**s).

The compromised package versions remained online for approximately two hours before being detected by developers and removed by **NPM!**s. As a result, the overall impact of the attack was limited.

4.2 Information theft attack - November 2025

The second attack considered in this study occurred on November 24 2025. Within a few hours, over 1,000 **NPM!**s packages were infected, all using the same technique; again relying on compromised maintainer accounts [Lak25].

The attack, known as *Sha1-Hulud*, introduced the Bun runtime as a dependency in the packages, adding a `preinstall` script (`node setup_bun.js`) along with a highly obfuscated file named `bun_environment.js`, with a size exceeding 10 MB [Tea25].

When executed, the script downloaded and ran TruffleHog to scan the local system, stealing sensitive information such as **NPM!**s tokens, **AWS!**s (**AWS!**s), **GCP!**s (**GCP!**s), and Azure credentials, and environment variables.

Chapter 5

Metrics

In questo capitolo, verranno illustrate le metriche scelte, il motivo per cui sono state prese in considerazione, e come sono state classificate.

Queste metriche sono state selezionate in base al threat modeling dei due attacchi analizzati in **??** (**??**); con l'obiettivo di identificare questi tipi di minacce.

Poichè non esiste uno standard universale per la nomenclatura e la classificazione di queste metriche, per questo lavoro è stato deciso di definirne una propria; basandosi su lavori precedentemente svolti. Nello specifico si è fatto riferimento al lavoro svolto da Ohm et al. [Ohm+20] e Cordey [Cor23].

Nel capitolo successivo, **??** (**??**) invece, verranno testate queste metriche su un piccolo dataset di pacchetti **NPM!**s con versioni malevole dei due attacchi analizzati in **??** (**??**). Oltre a spiegare come è stato costruito e scaricato questo dataset.

Sono state quindi individuate per questo lavoro cinque classi di metriche, che verranno descritte approfonditamente ognuna nella propria sezione apposita qui di seguito.

5.1 Evasion Metrics

La prima classe di metriche individuata è quella delle *Evasion Metrics*. Qui sono presenti le metriche che riguardano ...

Di seguito, in ogni sottosezione, sono riportate le metriche di questa classe.

5.1.1 Obfuscation patterns count

Questa prima metrica segnala la presenza e riporta il numero di occorrenze di valori esadecimali nel codice. In particolare, vengono considerati questi valori e variabili esadecimali presi singolarmente o se questi sono usati insieme a determinate strutture di codice; quali ad esempio parseInt, blocchi try-catch, assegnazioni di costanti e chiamate di funzioni.

Questa metrica è stata scelta in quanto l'offuscamento del codice è presente in entrambi gli attacchi analizzati in **??** (**??**); e in generale l'uso di valori esadecimali è una tecnica comune per offuscare il codice malevolo.

5.1.2 Platform detections count

Questa metrica segnala la presenza e conta il numero di controlli sul sistema operativo

Come ad esempio process.platform() == 'win32' o anche platform === "linux" (win(32—64—dows)—li
e .arch()

Il secondo tipo di attacco analizzato in **??** (**??**), ruba informazioni sensibili, l'idea quindi per questa metrica era quella di ...

5.2 Payload delivery execution Metrics

Chapter 6

Methodology

In the work conducted by Weissberg et al. [Wei+25], it is argued that often simple traditional code metrics are enough to achieve results similar to, if not better than, those obtained with complex LLM.

Chapter 7

Results

Chapter 8

Conclusion

Bibliography

[Cor23] Sean Cordey. *Software supply chain attacks: An illustrated typological review*. Tech. rep. ETH Zurich, 2023.

[GAH23] Betul Gokkaya, Leonardo Aniello, and Basel Halak. *Software supply chain: review of attacks, risk assessment strategies and security controls*. 2023. arXiv: 2305.14157 [cs.CR]. URL: https://arxiv.org/abs/2305.14157.

[Guo+24] Wenbo Guo, Zhengzi Xu, Chengwei Liu, Cheng Huang, Yong Fang, and Yang Liu. "An Empirical Study of Malicious Code In PyPI Ecosystem". In: *Proceedings of the 38th IEEE/ACM International Conference on Automated Software Engineering*. ASE '23. Echternach, Luxembourg: IEEE Press, 2024, pp. 166–177. ISBN: 9798350329964. DOI: 10.1109/ASE56229.2023.00135. URL: https://doi.org/10.1109/ASE56229.2023.00135.

[Hh25] Asaf Henig and Cameron hyde. *Breakdown: Widespread npm Supply Chain Attack Puts Billions of Weekly Downloads at Risk*. https://www.paloaltonetworks.com/blog/cloud-security/npm-supply-chain-attack/. 2025.

[Hua+24a] Cheng Huang, Nannan Wang, Ziyan Wang, Siqi Sun, Lingzi Li, Junren Chen, Qianchong Zhao, Jiaxuan Han, Zhen Yang, and Lei Shi. *DONAPI: Malicious NPM Packages Detector using Behavior Sequence Knowledge Mapping*. 2024. arXiv: 2403.08334 [cs.CR]. URL: https://arxiv.org/abs/2403.08334.

[Hua+24b] Yiheng Huang, Ruisi Wang, Wen Zheng, Zhuotong Zhou, Susheng Wu, Shulin Ke, Bihuan Chen, Shan Gao, and Xin Peng. "SpiderScan: Practical Detection of Malicious NPM Packages Based on Graph-Based Behavior Modeling and Matching". In: *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*. ASE '24. Sacramento, CA, USA: Association for Computing Machinery, 2024, pp. 1146–1158. ISBN: 9798400712487. DOI: 10.1145/3691620.3695492. URL: https://doi.org/10.1145/3691620.3695492.

[Lak25] Ravie Lakshmanan. *Second Sha1-Hulud Wave Affects 25,000+ Repositories via npm Preinstall Credential Theft*. https://thehackernews.com/2025/11/second-sha1-hulud-wave-affects-25000.html. 2025.

[Li+24] Siheng Li, Cheng Yang, Taiqiang Wu, Chufan Shi, Yuji Zhang, Xinyu Zhu, Zesen Cheng, Deng Cai, Mo Yu, Lemao Liu, Jie Zhou, Yujiu Yang, Ngai Wong, Xixin Wu, and Wai Lam. *A Survey on the Honesty of Large Language Models*. 2024. arXiv: 2409.18786 [cs.CL]. URL: https://arxiv.org/abs/2409.18786.

[Mat25] Nicolas Matthee. *The npm Supply Chain Attack of September 2025*. https://www.prventi.com/blog/the-npm-supply-chain-attack-of-september-2025. 2025.

[Ohm+20] Marc Ohm, Henrik Plate, Arnold Sykosch, and Michael Meier. *Backstabber's Knife Collection: A Review of Open Source Software Supply Chain Attacks*. 2020. arXiv: 2005.09535 [cs.CR]. URL: https://arxiv.org/abs/2005.09535.

[Sae+25] Mohammadreza Saeidi, Ethan Thoma, Raula Kula, and Gema Rodríguez-Pérez. *What About Our Bug? A Study on the Responsiveness of NPM Package Maintainers*. Nov. 2025. DOI: 10.48550/arXiv.2511.04986.

[Tea25] HelixGuard Team. *Shai-Hulud Returns: Over 1K NPM Packages and 27K+ Github Repos infected via Fake Bun Runtime Within Hours*. https://helixguard.ai/blog/malicious-sha1hulud-2025-11-24/. 2025.

[Wan+25] Jian Wang, Zhen Li, Jixiang Qu, Deqing Zou, Shouhuai Xu, Ziteng Xu, Zhenwei Wang, and Hai Jin. "MalPacDetector: An LLM-based Malicious NPM Package Detector". In: *IEEE Transactions on Information Forensics and Security* PP (Jan. 2025), pp. 1–1. DOI: 10.1109/TIFS.2025.3580336.

[Wei+25] Felix Weissberg, Lukas Pirch, Erik Imgrund, Jonas Möller, Thorsten Eisenhofer, and Konrad Rieck. "LLM-based Vulnerability Discovery through the Lens of Code Metrics". In: *arXiv preprint arXiv:2509.19117* (2025).

[Zah+25] Nusrat Zahan, Philipp Burckhardt, Mikola Lysenko, Feross Aboukhadijeh, and Laurie Williams. *Leveraging Large Language Models to Detect npm Malicious Packages*. 2025. arXiv: 2403.12196 [cs.CR]. URL: https://arxiv.org/abs/2403.12196.

[Zah+22] Nusrat Zahan, Thomas Zimmermann, Patrice Godefroid, Brendan Murphy, Chandra Maddila, and Laurie Williams. "What are weak links in the npm supply chain?" In: *Proceedings of the 44th International Conference on Software Engineering: Software Engineering in Practice*. ICSE-SEIP '22. Pittsburgh, Pennsylvania: Association for Computing Machinery, 2022, pp. 331–340. ISBN: 9781450392266. DOI: 10.1145/3510457.3513044. URL: https://doi.org/10.1145/3510457.3513044.

[Zha+25] Junan Zhang, Kaifeng Huang, Yiheng Huang, Bihuan Chen, Ruisi Wang, Chong Wang, and Xin Peng. "Killing Two Birds with One Stone: Malicious Package Detection in NPM and PyPI using a Single Model of Malicious Behavior Sequence". In: *ACM Transactions on Software Engineering and Methodology* 34.4 (Apr. 2025), pp. 1–28. ISSN: 1557-7392. DOI: 10.1145/3705304. URL: http://dx.doi.org/10.1145/3705304.

[Zim+19] Markus Zimmermann, Cristian-Alexandru Staicu, Cam Tenny, and Michael Pradel. "Smallworld with high risks: a study of security threats in the npm ecosystem". In: *Proceedings of the 28th USENIX Conference on Security Symposium*. SEC'19. Santa Clara, CA, USA: USENIX Association, 2019, pp. 995–1010. ISBN: 9781939133069.